PySpark Practical Assessment | Set 4

Streaming Platform Analytics • Google Colab • Participant Handout • No solutions

Scenario: You are working with a fictional video-streaming platform. Analyze viewing behavior, content popularity, device performance, subscriber plans, and playback quality using PySpark. All records are synthetic.

Assessment scope: DataFrames, filtering, joins, aggregations, null handling, conditional logic, dates, windows, union and deduplication, lazy execution, caching, persistence, partitioning, and Parquet output.

Provided datasets

File	Records	Purpose
watch_sessions_80000.csv	80,000	Primary viewing sessions
watch_sessions_extra_10000.csv	10,000	Additional batch; 600 exact duplicates
subscriber_master_2400.csv	2,400	Region, plan, signup, account status
content_catalog_120.csv	120	Title, genre, duration, language
device_master_8.csv	8	Device type, platform, screen size
subscription_plans_4.csv	4	Monthly fees, streaming limits

Expected validation: 90,000 rows after union; 89,400 unique `session_id` values after deduplication. The main session dataset has 80,000 rows.

Session columns: session_id, subscriber_id, content_id, device_id, session_start, watch_minutes, completion_pct, buffering_seconds, stream_quality, session_status, data_used_mb, user_rating.

Google Colab setup

Install PySpark if needed, upload the six CSVs using the file browser, and load them as Spark DataFrames.

```
!pip -q install pyspark
from google.colab import files
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window
from pyspark import StorageLevel
spark = SparkSession.builder.appName("StreamingSet4").getOrCreate()
files.upload()  # Select all six CSV files

def read_csv(name):
    return spark.read.csv(name, header=True, inferSchema=True)

sessions_df = read_csv("watch_sessions_80000.csv")
extra_df = read_csv("watch_sessions_extra_10000.csv")
subscribers_df = read_csv("subscriber_master_2400.csv")
content_df = read_csv("content_catalog_120.csv")
devices_df = read_csv("device_master_8.csv")
plans_df = read_csv("subscription_plans_4.csv")

print("Main sessions:", sessions_df.count())
sessions_df.printSchema()
sessions_df.show(5, truncate=False)
```

Part A - DataFrame operations

01. Display the schema, row count, and first 10 records of sessions_df. Identify numeric, string, and timestamp fields.

02. Select session_id, subscriber_id, content_id, watch_minutes, and session_status. Filter completed sessions with watch_minutes greater than 60.

03. Create watch_hours from watch_minutes and a high_engagement flag for sessions with at least 75% completion. Preserve null completion values.

04. Sort sessions by data_used_mb descending and display the top 20 records.

Part B - Data quality and conditional transformations

05. Count missing values in subscriber_id, device_id, completion_pct, buffering_seconds, and user_rating.

06. Create a new DataFrame that replaces missing buffering_seconds with zero. Do not modify the original DataFrame.

07. Create another DataFrame that excludes sessions with missing device_id. Compare record counts.

08. Use when() and otherwise() to label completion_pct as Low (below 30), Medium (30 to below 90), High (90 or more), or Unknown (null).

Part C - Joins and aggregations

09. Join sessions with the content catalog and calculate session counts and total watch_minutes by genre.

10. Join sessions with the device master and calculate average buffering_seconds and average watch_minutes by device_type.

11. Join sessions, subscribers, and plans. Calculate completion rate (completed sessions / all sessions) by plan_name, excluding null subscriber IDs.

12. Using subscriber_master and subscription_plans only, estimate monthly recurring revenue by plan for Active subscribers. Count each subscriber once; do not multiply fees by viewing sessions.

13. Identify sessions whose non-null subscriber_id or device_id has no matching master record. Report unmatched counts for each key.

Part D - Date and window functions

14. Convert session_start to timestamp. Calculate monthly session counts and total watch_minutes for 2026.
15. Join subscriber signup dates and calculate days between signup_date and each viewing session. Display the earliest 10 viewing sessions.
16. Classify sessions as Weekday or Weekend using the session date, then compare average watch_minutes.
17. Aggregate total watch_minutes by genre and content_title; use dense_rank() to identify the top three titles within each genre.
18. Use row_number() to retrieve the most recent session for each non-null subscriber_id.
19. Use lag() to compare each subscriber's watch_minutes with their previous session, ordered by session_start and session_id.

Part E - Union, lazy execution, caching and partitioning

20. Combine both session batches using unionByName(). Count rows and remove duplicate session_id values. Compare the totals.
21. Build a lazy pipeline with filter(), join(), withColumn(), groupBy(), and orderBy(). Inspect explain() before calling an action.
22. Cache completed sessions, run count() and an aggregation twice, and compare timings. Explain when the cache is populated.
23. Persist sessions with stream_quality = "4K" using MEMORY_AND_DISK. Execute two actions and then call unpersist().
24. Repartition sessions into 8 partitions and display row counts per partition using spark_partition_id().
25. Starting from 8 partitions, compare coalesce(2) and repartition(2) with explain(). Write a chosen output DataFrame to Parquet and inspect the part files.

Final capstone - Streaming performance report

1. Union the two batches and deduplicate by session_id.
2. Clean relevant nulls without discarding every incomplete record.
3. Join subscribers, content, devices, and plans as needed.
4. Produce monthly metrics by region and genre: session count, watch hours, completion rate, average buffering, and total data usage.
5. Rank the top three content titles by watch hours within each genre.
6. Cache the final aggregated DataFrame, reuse it for two actions, and unpersist it.
7. Save the final report as Parquet with 3 output partitions.

Submission: One Google Colab notebook containing code, key outputs, execution-plan observations, and the final Parquet output folder. Solutions are not included in this handout.